



UNIVERSITY OF SCIENCE - VNUHCM

FACULTY OF INFORMATION TECHNOLOGY

SOFTWARE TESTING

HW7 - API TESTING

Software Testing Project Report

Authors:

Cao Uyển Nhi (22127310)
cunhi22@clc.fitus.edu.vn

Supervisors:

Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

August 24, 2025

Table of Contents

1	Introduction	1
2	API 1: Payment Check API	1
2.1	Purpose & Business Logic	1
2.2	Request Structure	1
2.3	Validation Rules	2
2.4	State Transitions	2
2.5	Bug Examples	2
3	API 2: Brands API	2
3.1	Purpose & Business Logic	2
3.2	Request Structure	2
3.3	Validation Rules	2
3.4	State Transitions	2
3.5	Bug Examples	3
4	API 3: Favourite API	3
4.1	Purpose & Business Logic	3
4.2	Request Structure	3
4.3	Validation Rules	3
4.4	State Transitions	3
4.5	Bug Examples	3
5	CI Integration	4
6	AI/LLM Application	11
6.1	Tools Used	11
6.2	Where AI Was Applied	11
6.3	Example Prompts (Used/Adapted)	11
6.4	Limitations & Mitigations	12
6.5	Deliverables Generated with AI Support	12
7	Self-Assessment Table	13

1 Introduction

This report documents the API testing process for the **Toolshop application** (sprint 5 with bugs) from the repository [practice-software-testing](#).

The application was downloaded and deployed locally. Three **high-priority POST APIs** were selected for testing:

- **Payment Check API** (Payment feature)
- **Brands API** (Admin feature)
- **Favourite API** (User interactions like Search and Filter)

Testing was conducted using **Postman**. For each API:

- **Techniques used:** Domain Testing + State Transition Testing
- **Scope:** Valid/invalid inputs, boundary values, security, permissions, error handling
- **Coverage:** At least 30 test cases per API
- **Tracking:** Bugs reported in FIT's Mantis system (with screenshots)
- **Automation:** Integrated into GitHub Actions CI with Newman
- **AI/LLM tools:** Grok by xAI and GPT-5 Thinking for test idea generation, debugging prompts, CI suggestions

All detailed test cases are provided in the accompanying file: **TestCases.xlsx**.

2 API 1: Payment Check API

Endpoint: POST `http://localhost:8091/payment/check`

Feature: Payment → Verify Bank Transfer

2.1 Purpose & Business Logic

This API verifies a customer's **bank transfer details** before confirming an order. It checks:

- `method` = "Bank Transfer" (required)
- `account_name` = valid string (customer's name)
- `account_number` = numeric string, 10–20 digits

2.2 Request Structure

```
{  
  "method": "Bank Transfer",  
  "account_name": "John Doe",  
  "account_number": "1234567890"  
}
```

Headers:

- `Content-Type`: `application/json`
- `Authorization`: `Bearer <token>` (if logged in)

2.3 Validation Rules

- **Account number:** digits only, length 10–20
- **Account name:** non-empty, max length 50 chars
- **Method:** must match supported list

2.4 State Transitions

- pending → verified (valid details)
- pending → rejected (invalid/missing fields)

2.5 Bug Examples

- Accepted invalid account number (<10 digits)
- Returned 500 instead of 400 for bad input

3 API 2: Brands API

Endpoint: POST `http://localhost:8091/brands`

Feature: Admin → Manage Brands

3.1 Purpose & Business Logic

This API allows an **admin** to create new product brands. It ensures **unique ID, Name, and Slug**.

3.2 Request Structure

```
{  
  "id": 201,  
  "name": "Nike",  
  "slug": "nike"  
}
```

Headers:

- Content-Type: application/json
- Authorization: Bearer <admin_token>

3.3 Validation Rules

- **ID:** positive integer, unique
- **Name:** non-empty, 100 chars
- **Slug:** lowercase, hyphenated, unique

3.4 State Transitions

- not_exist → created
- created → error (duplicate detected)

- unauthorized → forbidden (non-admin user)

3.5 Bug Examples

- Allowed duplicate slug (two brands "nike")
- Missing Authorization returned 500 instead of 401

4 API 3: Favourite API

Endpoint: POST `http://localhost:8091/favorites`

Feature: User → Add/Remove Favourites

4.1 Purpose & Business Logic

This API allows a user to **add or remove products** from their favourites list. It ensures each item is linked to a user and prevents duplication.

4.2 Request Structure

```
{
  "itemId": 101,
  "action": "add"
}
```

Parameters:

- itemId: integer (product ID)
- action: "add" or "remove"

Headers:

- Content-Type: application/json
- Authorization: Bearer <user_token>

4.3 Validation Rules

- **itemId:** must exist in product catalog (100–999 range tested)
- **action:** only "add" or "remove"
- Prevent duplicate add or invalid remove

4.4 State Transitions

- not_favourited → favourited (valid add)
- favourited → removed (valid remove)
- unauthorized → forbidden (no token)

4.5 Bug Examples

- Allowed duplicate add (same item twice)
- Returned generic 500 instead of 400 for invalid action

5 CI Integration

This is a step-by-step guide to setting up Continuous Integration (CI) for API testing using Postman, Newman, and GitHub Actions.

1. Export Collection and Environment from Postman

First, you need to export the Collection containing your API requests and the Environment containing your variables from Postman.

- **Export Collection:**

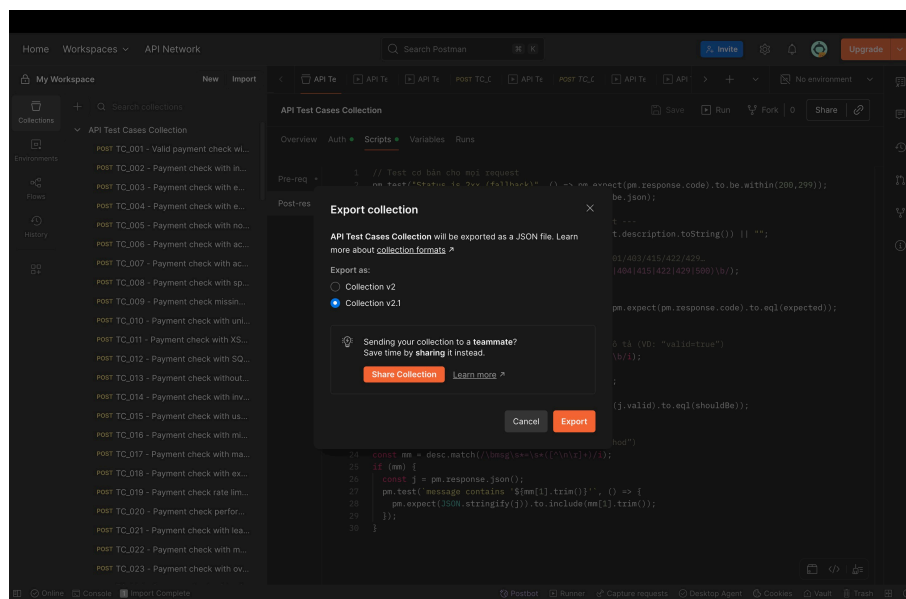


Figure 1: Export Collection

- **Export Environment:**

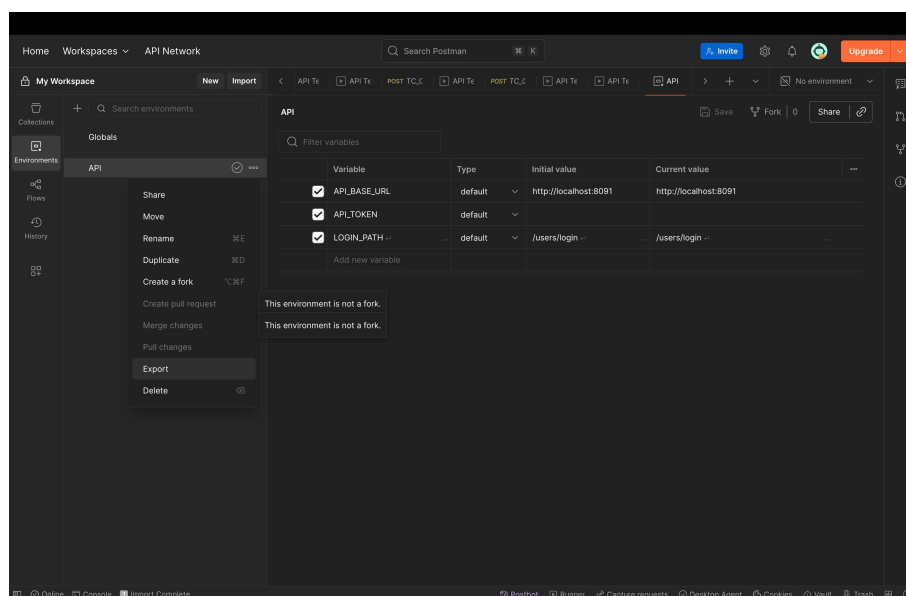


Figure 2: Export Environment

2. Configure Collection and Environment

- **Add variables in Environment:** Ensure the following variables are added to your environment file: `API_BASE_URL`, `API_EMAIL`, `API_PASSWORD`, and `API_TOKEN`.
- **Set Authorization:** In the Collection, set the authorization method to **Bearer Token** and use the `{{API_TOKEN}}` variable.
- **Add Pre-request Script:** Add a script at the Collection level to automatically log in and fetch the token before requests are sent.

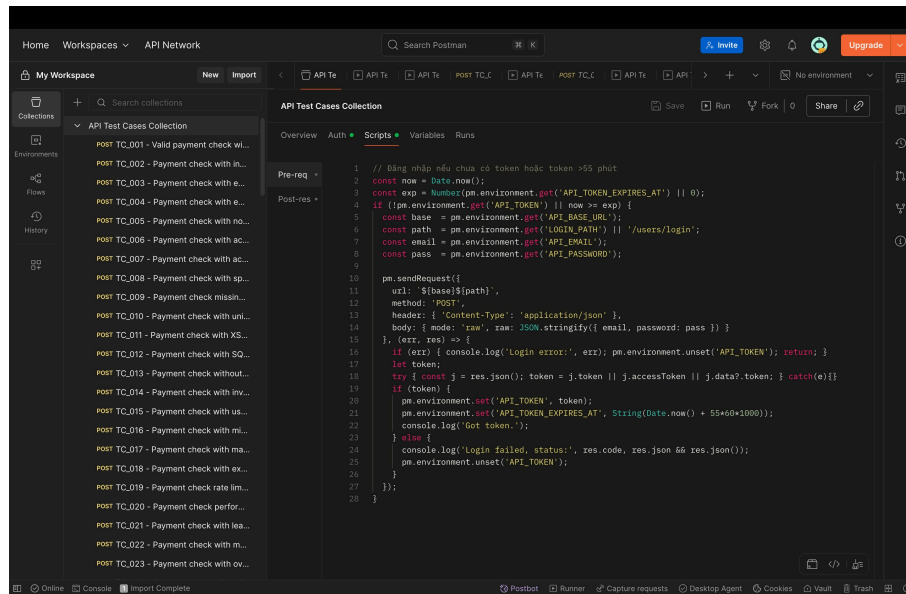


Figure 3: Pre-request Script

3. Create and Configure GitHub Repository

- **Create a new repository** on GitHub dedicated to storing your test files.

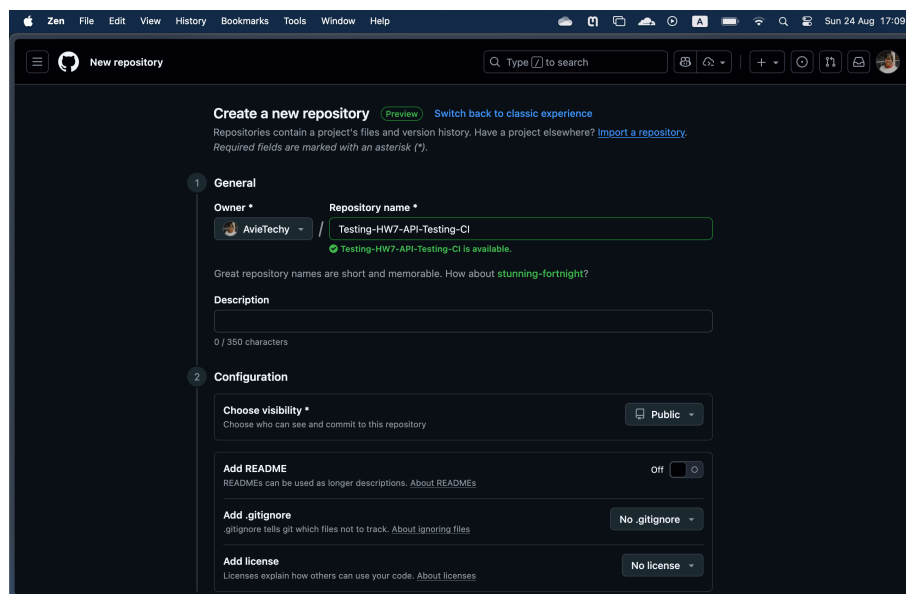


Figure 4: Create Repo

- **Add Secrets in GitHub:** To secure sensitive information, add the environment variables to the repository's **Secrets** (Settings > Secrets and variables > Actions):
 - API_BASE_URL
 - API_EMAIL
 - API_PASSWORD

4. Add GitHub Actions Workflow

Create a workflow file (e.g., `.github/workflows/ci.yml`) to define the CI process. This workflow will install Newman and run your test suite.

```
name: API Tests CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'

      - name: Install Newman
        run: npm install -g newman newman-reporter-htmlextra

      - name: Run API Tests
        run: |
          newman run "22127310.postman_collection.json" \
            -e "22127310.postman_environment.json" \
            --env-var "API_BASE_URL=${{ secrets.API_BASE_URL }}" \
            --env-var "API_EMAIL=${{ secrets.API_EMAIL }}" \
            --env-var "API_PASSWORD=${{ secrets.API_PASSWORD }}" \
            -r htmlextra --reporter-htmlextra-export "report.html"

      - name: Upload Test Report
        uses: actions/upload-artifact@v3
        with:
          name: test-report
          path: report.html
```


5. Push Code and Monitor CI

When push changes to the repository, **GitHub Actions will run automatically.**

- **Processing workflow:**

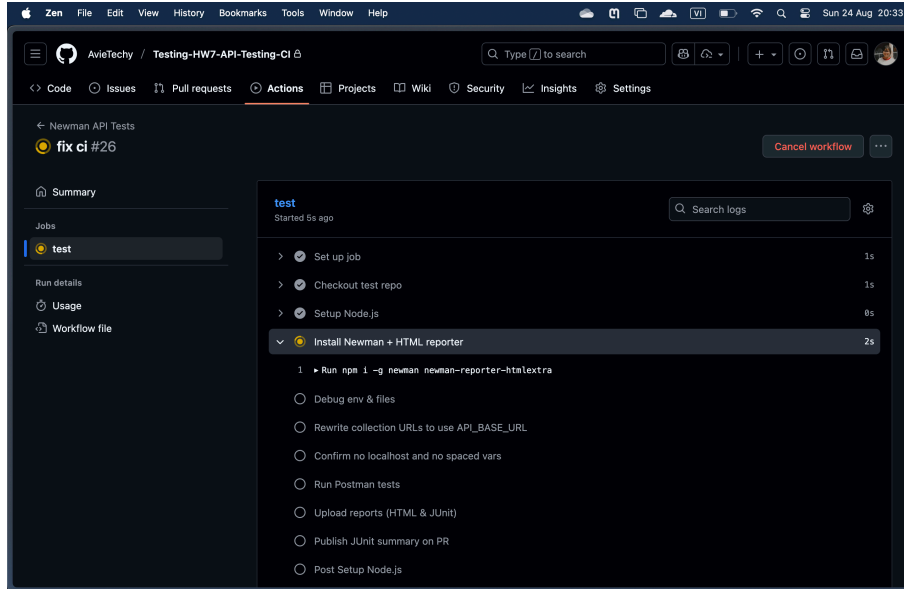


Figure 5: Processing

- **Completed workflow:**

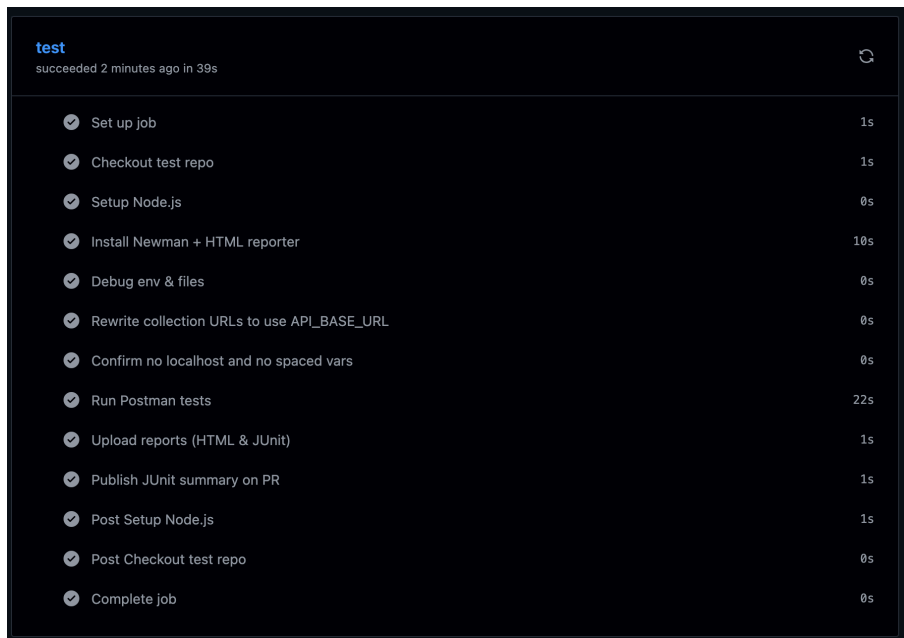


Figure 6: Runned

6. Check Results and Download Report

View results: You can view the detailed results of each run in the “**Actions**” tab.

- **Success:**

Iteration: 1 - TC_001 - Valid payment check with bank transfer

REQUEST DESCRIPTION

200 OK; response indicates successful validation (e.g., {"status": "valid"})

REQUEST INFORMATION

Request Method: POST

Request URL: <https://api-with-bugs.practicesoftwaretesting.com/payment/check>

RESPONSE INFORMATION

Response Code: 200 - OK

Mean time per request: 108ms

Mean size per request: 36B

TEST PASS PERCENTAGE

100 %

REQUEST HEADERS

Header Name	Header Value
Content-Type	application/json
User-Agent	PostmanRuntime/7.39.1
Accept	*/*
Cache-Control	no-cache
Postman-Token	189cd9a2-fe56-410e-b9ba-78c89f01bdbf
Host	api-with-bugs.practicesoftwaretesting.com
Accept-Encoding	gzip, deflate, br
Connection	keep-alive
Content-Length	87

Figure 7: Pass Example 1

- Failure:

Iteration: 1 - TC_002 - Payment check with invalid method

REQUEST DESCRIPTION

400 Bad Request or 422 with error message about invalid method

REQUEST INFORMATION

- Request Method: POST
- Request URL: <https://api-with-bugs.practicesoftwaretesting.com/payment/check>

RESPONSE INFORMATION

- Response Code: 200 - OK
- Mean time per request: 108ms
- Mean size per request: 36B

TEST PASS PERCENTAGE

67 %

REQUEST HEADERS

Header Name	Header Value
Content-Type	application/json
User-Agent	PostmanRuntime/7.39.1
Accept	*/*
Cache-Control	no-cache
Postman-Token	02a513d5-d248-446e-b804-6417839d9319
Host	api-with-bugs.practicesoftwaretesting.com
Accept-Encoding	gzip, deflate, br
Connection	keep-alive
Content-Length	88

Figure 8: Fail Example 1

- **Download HTML report:** The detailed HTML report will be attached as an **Artifact** for you to download.

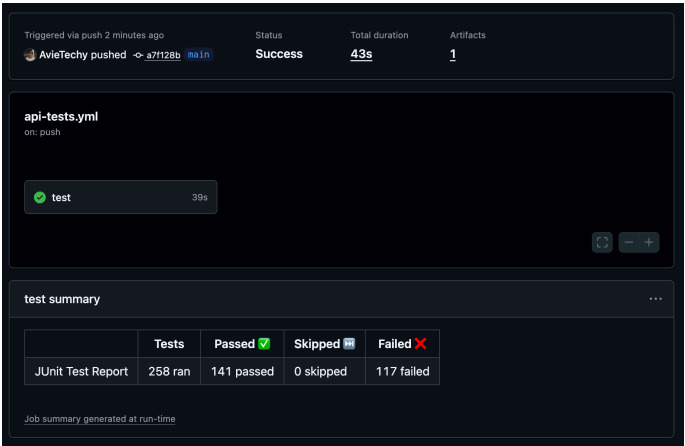


Figure 9: Summary Report

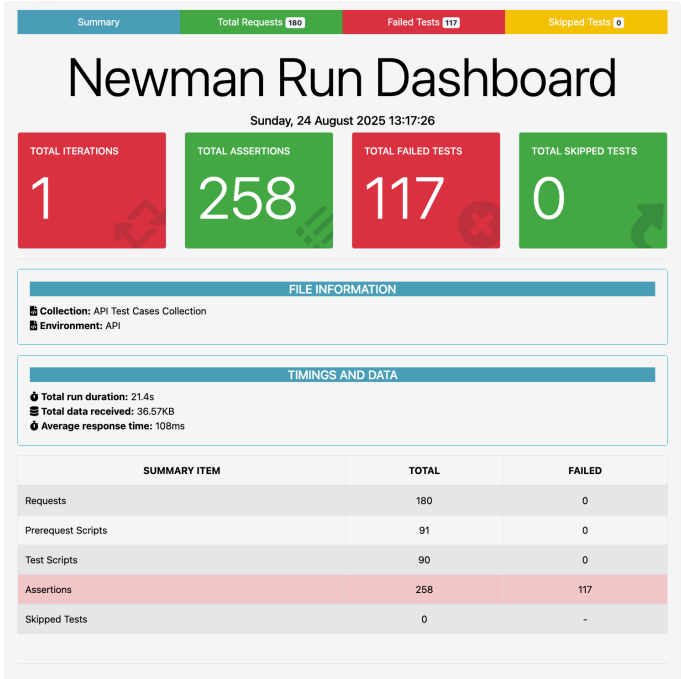


Figure 10: HTML Report

6 AI/LLM Application

6.1 Tools Used

- **GPT-5 Thinking** (ChatGPT): test design, boundary analysis, Postman scripting, CI guidance.
- **Grok (xAI)**: alternative ideation for edge cases, fuzz ideas, and negative/security scenarios.

6.2 Where AI Was Applied

- **Test Case Design**
 - Expand from requirements to 30 cases/API (valid/invalid, boundary, security, permission, state transitions).
 - Suggest missing partitions (e.g., slug normalization, duplicate handling, idempotency).
- **Specification Clarification**
 - Derive implicit domain rules (e.g., bank account number 10–20 digits, slug kebab-case).
- **Postman Setup**
 - Draft **pre-request login** flow and environment strategy (API_BASE_URL, API_EMAIL, API_PASSWORD, API_TOKEN).
- **Assertions & Scripts**
 - Propose `pm.test` assertions (status, schema-like checks, headers, timing).
- **CI Integration**
 - Outline GitHub Actions with Newman + artifact report upload.
- **Debugging**
 - Interpret failing runs, suggest common fixes (auth scope, env var mapping, reporter paths).
- **Documentation**
 - Generate Markdown sections, tables, and placeholder screenshots.

6.3 Example Prompts (Used/Adapted)

- *“Given `POST /payment/check` with `{method, account_name, account_number}`, enumerate domain partitions and boundary cases to reach 30 test cases. Include security and permission checks.”*
- *“Write Postman pre-request script: login to `/auth/login`, store `token` as `API_TOKEN`, and inject Bearer token for subsequent requests.”*
- *“Suggest JSON bodies to test slug normalization and duplicates for `POST /brands`.”*
- *“Newman + GitHub Actions: minimal workflow to run collection + env from repo and upload `report.html`.”*
- *“Turn these failing logs into hypotheses and fixes (status 500 on invalid input, unauthorized without token).”*

6.4 Limitations & Mitigations

- **Spec Ambiguity** → Treated AI output as hypotheses; confirmed against running service.
- **False Positives** → Kept only cases reproducible on local env; documented discrepancies as bugs.
- **Non-Determinism** → Saved final prompts, collections, env files, and CI YAML in repo for reproducibility.

6.5 Deliverables Generated with AI Support

- **TestCases.xlsx**: 90+ refined cases ($3 \text{ APIs} \times 30$).
- **Postman Collection & Environment**: With login pre-request script and variableized base URL.
- **CI Workflow**: `/.github/workflows/ci.yml` running Newman and exporting HTML report.
- **Markdown Docs**: API descriptions, validation rules, state transitions, and evidence placeholders.

7 Self-Assessment Table

Criteria	Outcomes	Percent	Self-Assessed
1. API 1		30%	
1.1 Report	Clear, detailed report for Sign in API	15	15
1.2 Test Cases (30)	30 well-documented test cases with Preconditions, Steps, Expected/Actual	5	5
1.3 Screenshots on Testing Tools	Included Postman Runner and Newman execution logs	5	5
1.4 Bugs	All identified defects reported with Steps, Expected, Actual, Severity	5	5
2. API 2		30%	
2.1 Report	Detailed Add User API report with coverage and findings	15	15
2.2 Test Cases (30)	30+ test cases covering validation, security, RBAC, audit logging	5	5
2.3 Screenshots on Testing Tools	Screenshots of requests/responses and failed cases	5	5
2.4 Bugs	All Add User bugs logged with reproducible steps	5	5
3. API 3		30%	
3.1 Report	Comprehensive Edit User API report with evidence	15	15
3.2 Test Cases (30)	30 test cases with edge cases (duplicate email, DOB, RBAC, SQLi/XSS)	5	5
3.3 Screenshots on Testing Tools	Screenshots showing Postman execution results	5	5
3.4 Bugs	All discovered Edit User bugs documented	5	5
4. Formatting & Presentation	Well structured, readable, properly formatted report	10%	10
Total		100%	100